

INARA Feature Engineering — Technical Notes

Pipeline Step 2 | CS 6140 · Machine Learning · Northeastern University · Spring 2026

Shantanu Wankhare · Bhalchandra Shinde · Asad Mulani

This document describes every decision made in `pipeline/steps/02_feature_engineer.py`: the train/val/test split, spectral Z-score normalisation, PCA feature reduction for the Random Forest baseline, and data quality checks. The deep model (1D CNN) uses normalised spectra directly and does not use PCA.

1. Overview

Step 2 runs **once** before Steps 3–5, producing all engineered artifacts in `inara_data/engineered/`. Because both the RF baseline and the 1D CNN load from these same files, they are guaranteed to train and evaluate on identical data splits — a prerequisite for a fair academic comparison.

Input	Output
spectra.npy (N, 12, 101) — raw CLIMA profiles	spectra_{train,val,test}.npy — Z-normalised
molecules.npy (N, 12) — log██ targets	molecules_{train,val,test}.npy — raw log██
	feat_{train,val,test}.npy — PCA (RF only)
	scaler.joblib, pca.joblib, feature_info.json
	{train,val,test}_indices.npy — split indices

2. Current Dataset

Property	Value
Total samples (this run)	25,000
Train split (70%)	17,500 samples
Validation split (15%)	3,750 samples
Test split (15%)	3,750 samples
Input shape	(N, 12, 101) — 12 CLIMA channels × 101 altitude levels
Target shape	(N, 12) — log██ surface volume mixing ratios
Split seed	42 (reproducible)
Full dataset size	3,112,620 samples (Zorzan et al. 2025)

██ The 25,000-sample run is a local development run. The full 3.1M-sample dataset is available on Northeastern Explorer HPC. All pipeline steps are designed to scale — only the extraction count and RF cap need adjusting in `pipeline/config.yaml`.

3. Train / Val / Test Split (70 / 15 / 15)

```
train_idx, val_idx, test_idx = split_indices(
    N, val_frac=0.15, test_frac=0.15, seed=42
)
```

Why this ratio?

- 70 % training gives the CNN sufficient samples (17,500 in this run; ~87,000 on 124k).
- 15 % test set (3,750 samples) is large enough for reliable per-molecule R^2 estimates.
- Identical test set for RF and CNN — directly comparable metrics, no cherry-picking.
- Seed 42 is fixed in config.yaml — reproducible across machines and reruns.
- Indices saved to disk so Steps 3–5 never re-split; data cannot leak between runs.

4. Spectral Z-score Normalisation

Each of the 12 CLIMA channels is normalised **independently** across the altitude axis:

```
x_norm[n, c, l] = (x[n, c, l] -  $\mu$ [c, l]) /  $\sigma$ [c, l]

#  $\mu$ ,  $\sigma$  computed on TRAINING SET ONLY, then applied to val and test
scaler = SpectraScaler()
spec_train = scaler.fit_transform(spectra[train_idx])
spec_val = scaler.transform(spectra[val_idx])
spec_test = scaler.transform(spectra[test_idx])
```

Design choice	Rationale
Per-channel, per-altitude-level	CLIMA channels span vastly different scales (temperature in K vs mixing ratios in [0,1]). Per-element normalisation is required.
Fit on train only	Fitting on val/test would leak their statistics into training — a form of data leakage. The scaler sees only training data.
std floor = 1e-8	Channels with near-zero variance (constant altitude levels) would cause $\div 0$. Any std < 1e-8 is replaced with the floor.
Saved as scaler.joblib	Steps 3–5 and the dashboard load the same scaler to ensure consistent normalisation at inference time.

5. PCA Feature Reduction (Random Forest only)

The RF baseline cannot process the raw (N, 12, 101) tensor. Feature engineering flattens and reduces it:

```
# Flatten: (N, 12, 101) → (N, 1212)
flat = spectra.reshape(N, -1).astype(np.float64) # float64 avoids matmul overflow

pca = PCA(n_components=300, svd_solver="full", whiten=False)
feat_train = pca.fit_transform(flat_train) # (N_tr, 300)
feat_val = pca.transform(flat_val) # (N_v, 300)
feat_test = pca.transform(flat_test) # (N_te, 300)
```

Parameter	Value	Rationale
n_components	300	Covers 100 % variance on the 25k run (data is rank-limited). On larger datasets captures ~95 % variance.
svd_solver	"full"	Deterministic, numerically stable. Avoids randomised approximation artefacts in transform().
whiten	False	Whitening divides by singular values; near-zero values cause $\div 0$. Not needed for RF — tree splits are robust to scale.

Parameter	Value	Rationale
dtype	float64	float32 matmul (17500 × 1212) @ (1212 × 300) can overflow on macOS Accelerate BLAS. float64 el

Why does the 1D CNN NOT use PCA?

The CNN processes the raw 2D tensor (12 × 101) directly. Convolutional layers learn their own spatial feature extraction — they capture non-linear altitude patterns that a linear PCA projection cannot represent. PCA is only needed to make the RF tractable.

6. Target Normalisation (1D CNN only)

The 12 molecular targets span very different log₁₀ ranges — NH is fixed at −40, while N ranges from −1.1 to −0.01. Without normalisation, the weighted MSE loss is dominated by molecules with large absolute values, destabilising early training.

```
# In pipeline/steps/04_train_deep.py (NOT in feature engineering)
mol_scaler = MoleculeScaler() # per-molecule Z-score
mol_train_scaled = mol_scaler.fit_transform(mol_train)
mol_val_scaled = mol_scaler.transform(mol_val)
mol_test_scaled = mol_scaler.transform(mol_test)

# After training, inverse-transform predictions before computing metrics:
test_pred_log10 = mol_scaler.inverse_transform(trainer.predict(test_loader))
test_df = compute_metrics(mol_test, test_pred_log10) # metrics in log10 space
```

Note: MoleculeScaler is fit on *training targets only* to avoid leakage. R²/RMSE/MAE are always reported in the original log₁₀ space for interpretability and comparability with the RF baseline.

7. Data Quality Report (Step 2 output)

After saving all engineered arrays, Step 2 runs a quality check and prints a summary table. This is the actual output from the 25k run:

Array	Shape	NaN	Inf	Min	Max	Status
spectra_train	(17500, 12, 101)	0	0	−68.12	132.28	OK
spectra_val	(3750, 12, 101)	0	0	−68.12	76.38	OK
spectra_test	(3750, 12, 101)	0	0	−31.51	139.48	OK
molecules_train	(17500, 12)	0	0	−40.00	−0.01	OK
molecules_val	(3750, 12)	0	0	−40.00	−0.01	OK
molecules_test	(3750, 12)	0	0	−40.00	−0.01	OK
feat_train	(17500, 300)	0	0	−289.71	167.16	OK
feat_val	(3750, 300)	0	0	−289.71	88.16	OK
feat_test	(3750, 300)	0	0	−134.37	255.91	OK

Target standard deviations (per molecule, training set)

Molecule	Std (log $\square\square$)	Learnable?	Notes
H $\blacksquare$ O	0.0002	LOW	Near-constant — R ² unreliable; excluded from mean R ²
CO $\blacksquare$	0.8442	YES	High variance — strong learnable signal
O $\blacksquare$	0.1709	YES	
O $\blacksquare$	0.1234	YES	Strong spectral signature — best-predicted molecule
CH $\blacksquare$	1.8232	YES	High variance — CNN shows strong improvement
N $\blacksquare$	0.0446	YES	Low variance — RF and CNN both struggle
N $\blacksquare$ O	0.1527	YES	
CO	1.3330	YES	Both models struggle — photochemical degeneracy
H $\blacksquare$	2.5344	YES	High variance — challenging for both models
H $\blacksquare$ S	0.3954	YES	
SO $\blacksquare$	0.1125	YES	
NH $\blacksquare$	0.0000	NO	CONSTANT — all samples at log-floor (−40). RF: R ² =1.0 (memorises floor). CNN: R ² =0.0. Excluded

8. Artifacts Saved to inara_data/engineered/

File	Shape / Type	Used by
spectra_train.npy	(17500, 12, 101) float32	Step 4 (CNN training)
spectra_val.npy	(3750, 12, 101) float32	Step 4 (CNN validation)
spectra_test.npy	(3750, 12, 101) float32	Step 4 (CNN test eval)
molecules_train.npy	(17500, 12) float32	Steps 3, 4
molecules_val.npy	(3750, 12) float32	Steps 3, 4
molecules_test.npy	(3750, 12) float32	Steps 3, 4, 5
feat_train.npy	(17500, 300) float64	Step 3 (RF training)
feat_val.npy	(3750, 300) float64	Step 3 (RF validation)
feat_test.npy	(3750, 300) float64	Step 3 (RF test eval)
scaler.joblib	SpectraScaler object	Dashboard, inference
pca.joblib	sklearn PCA object	Step 3, dashboard
train_indices.npy	(17500,) int64	Provenance
val_indices.npy	(3750,) int64	Provenance
test_indices.npy	(3750,) int64	Provenance
feature_info.json	JSON metadata	Dashboard, verification

9. Relevant Configuration (pipeline/config.yaml)

```
data:
  val_frac: 0.15
  test_frac: 0.15
  seed: 42

model:
  pca_components: 300 # components for RF PCA features

profiles:
  local:
    processed_dir: inara_data/processed
    engineered_dir: inara_data/engineered
    results_dir: results/processed
  hpc:
    processed_dir: /scratch/shinde.b/inara/processed
    engineered_dir: /scratch/shinde.b/inara/engineered
    results_dir: /scratch/shinde.b/inara/results/processed
```